

Replayable Risk-Limiting Audit Samplers And Stopping Checks

Gio Della-Libera

Draft — June 28, 2026

Abstract

Risk-limiting audits depend on two separable claims: the sample was selected from the declared ballot population by a public random process, and the sampled ballots provide enough evidence to stop or escalate under a named rule. This paper lands both an RCOUNT sampler-replay contract and a first stopping-check contract. A verifier recomputes the contest audit population from normalized CVR rows, replays a SHA-256 sampler from public seed material, checks sampled observations against CVR interpretations, binds reported margin metadata to public summaries, and verifies that the declared pass-or-escalate status follows from an explicit stopping method. The fixture pairs prove the replay, margin, observation, taxonomy, first statistical-risk surface, and first jurisdiction-adaptor record contracts.

1 Introduction

Risk-limiting audits (RLAs) are often described as statistical audits, but a large part of public trust sits one layer below the statistical test. Observers must be able to ask whether the sampled ballots were actually the ballots that the public seed and manifest should have selected.

RCOUNT therefore treats sampler replay as its own contract. The verifier does not need to trust a vendor user interface, an exported spreadsheet, or a hand-copied sample list. It recomputes the population and the draws from published package data.

This paper covers the first V.7 substrate slice. It does not claim that an RLA has stopped correctly. It claims that the sample list used by a later stopping rule is reproducible from public evidence.

2 Sampler Contract

The implemented audit record stores:

Field	Meaning
<code>audit_id</code>	Stable id for the audit round.
<code>contest_id</code>	Contest whose CVR rows define the audit population.
<code>jurisdiction_method_id</code>	Optional named jurisdiction adapter, such as <code>colorado-rule-25-comparison-v1</code> .
<code>risk_limit_ppm</code>	Risk limit in parts per million, avoiding floating-point drift.
<code>public_seed</code>	Public random seed, such as a dice-generated seed ceremony value.
<code>sampling_algorithm_id</code>	Versioned sampler name. Current value: <code>rcount-sha256-modulo-v1</code> .
<code>manifest_hash</code>	Hash of the sorted unique CVR ids for the contest.
<code>sample_size</code>	Number of draws in this replay round.
<code>sample_draws</code>	Published draw index and selected CVR id pairs.

The population is the sorted unique set of `cvr_id` values for the target contest. Draw i hashes the domain prefix, manifest hash, public seed, contest id, risk limit, draw index, and algorithm id. The first eight digest bytes are interpreted as a little-endian integer and reduced modulo the population length.

This modulo sampler is intentionally simple and versioned. A future statutory or state-specific sampler can live beside it under a different algorithm id.

3 Implemented Surface

The RCOUNT implementation adds audit-round and sample-draw records to `rcount-core`. The package verifier now runs `verify_rla_sampler_replay` after CVR reconciliation, so a package can only pass when the declared RLA sample draws replay.

The second implemented layer adds sample observations and a conservative rule: any mark mismatch forces escalation. Every draw must have one observation. The observation must identify the sampled CVR id, and its observed selections and residual flags are compared to the normalized CVR row. If the number of discrepancies is no greater than `max_discrepancies`, the computed status is `pass`; otherwise it is `escalate`.

Discrepancies are now first-class records. The verifier classifies mismatches as selection, residual, combined selection-and-residual, or wrong-CVR observations. When a package declares a discrepancy list, the declared draw ids, CVR ids, and kinds must match the computed taxonomy exactly.

Margin metadata is also bound to the public summary before statistical stopping math is attempted. The audit record names a winner selection, loser selection, reported votes, reported margin, and diluted-margin denominator. The verifier checks those fields against the canvassed jurisdiction total.

The first named statistical method is `comparison-margin-threshold-v1`. It is intentionally conservative and simple: it recomputes a risk estimate from sample size and reported margin, adds a large penalty for declared discrepancies, checks the declared risk estimate, and only permits `pass` when the estimate is at or below the audit’s risk limit and discrepancy count is within the declared threshold.

The first jurisdiction adapter is `colorado-rule-25-comparison-v1`. It does not claim full Colorado certification compliance. It verifies the RCOUNT-side contract needed by a

Colorado-style comparison audit fixture: a 20 digit public seed, the named SHA-256 sampler, margin metadata, a comparison-margin stopping method, declared risk estimate, and declared pass/escalate status.

The second jurisdiction adapter is `california-public-rla-v1`. It targets a different public-contract layer: the audit record must name a California-style ballot manifest format, public audit software identity, public source URL, margin metadata, and declared audit status. This models the open-algorithm and manifest-shape requirements before any later statutory risk calculation is added.

Check	Failure caught
<code>rla_sampler_replay</code>	Duplicate audit ids, invalid risk limits, empty or wrong-sized samples, unsupported algorithm ids, manifest hash drift, and edited sample draws.
<code>rla_margin_metadata</code>	Winner/loser selection ids, reported votes, reported margin, and diluted-margin denominator drift from the jurisdiction summary.
<code>rla_stopping_rule</code>	Missing or duplicate observations, wrong sampled CVRs, declared discrepancy taxonomy drift, risk-estimate drift, and declared stop/escalate outcomes that disagree with observed discrepancies.
<code>rla_jurisdiction_adapter</code>	Unsupported jurisdiction methods; Colorado-style records missing seed, sampler, margin, or stopping-risk fields; California-style records missing public software or manifest-format metadata.
<code>cvr_summary_total</code>	CVR rows still reconcile to the public summaries before they are used as the audit population.
<code>source_hash_match</code>	Source evidence bytes still match the declared source index.

The audit transcript maps sampler failures to `rla_sampler_replay` and stopping failures to `rla_stopping_rule`. Jurisdiction adapter failures surface as `rla_jurisdiction_adapter`, making all three layers visible through the same CLI surface as prior RCOUNT checks.

4 Fixtures

The positive fixture is `rla-replay`. It extends the V.6 CVR summary fixture with one audit round for `syn-2024-mayor`. The package contains 140 CVR contest rows, a 5 percent risk limit encoded as 50000 ppm, a 20-digit public seed, and 12 replayed sample draws.

Path	Role
<code>audits/rla.ndjson</code>	RLA replay records and sample draws.
<code>normalized/cvr.ndjson</code>	Contest CVR rows used to derive the audit population.
<code>normalized/summaries.ndjson</code>	Public totals reconciled against CVR rows.
<code>sources/source-index.json</code>	Source hash evidence for the synthetic export.
<code>transcripts/verify-transcript.json</code>	CLI verifier transcript.

The negative fixture is **bad-rla-replay**. It changes the first published sample draw to **cvr:P-999:999**. The manifest hash and source hash still verify, but sampler replay recomputes draw 0 as **cvr:P-001:022** and fails the package.

The stopping fixture is **rla-stopping**. It uses the same replayed sample but adds one observation per draw. Each observation repeats the sampled CVR id and records the observed contest interpretation. With zero discrepancies and **max_discrepancies = 0**, the declared status **pass** verifies.

The stopping negative fixture is **bad-rla-stopping**. It leaves sampler replay intact, but changes one observed interpretation. The computed status is **escalate**, so the declared **pass** fails under **rla_stopping_rule**.

The discrepancy fixture is **rla-discrepancy**. It contains the same observed mark mismatch as **bad-rla-stopping**, but declares the mismatch as a selection discrepancy and declares **escalate**. The package passes because the observation, discrepancy taxonomy, and status agree.

The taxonomy negative fixture is **bad-rla-discrepancy**. It declares the same observed mark mismatch as a residual discrepancy. The verifier recomputes a selection discrepancy and fails the taxonomy check.

The margin fixture is **rla-margin**. It adds reported winner votes, loser votes, reported margin, and diluted-margin denominator to the audit record. Those fields are recomputed from the canvassed jurisdiction summary before the audit passes.

The margin negative fixture is **bad-rla-margin**. It changes the reported margin while leaving the public summary unchanged. The package fails under the margin metadata check.

The statistical fixture is **rla-statistical**. It keeps the replayed sample, observations, and margin metadata, then names **comparison-margin-threshold-v1** as the stopping method. The audit record declares the recomputed risk estimate in parts per million. The verifier recomputes that value from sample size, reported margin, and discrepancy count, then accepts the declared **pass** only when both the discrepancy threshold and risk limit are satisfied.

The statistical negative fixture is **bad-rla-statistical**. It leaves the sample, observations, margin metadata, and declared status unchanged, but edits the declared risk estimate by one part per million. The package therefore fails under **rla_stopping_rule**; the transcript exposes the declared and computed risk estimates side by side.

The jurisdiction fixture is **colorado-rla**. It adds a Colorado-style method id to the statistical fixture. The added adapter check verifies that the public seed is exactly 20 decimal digits and that the record carries the expected sampler, margin, stopping method, risk estimate, and declared status fields.

The jurisdiction negative fixture is **bad-colorado-rla**. It keeps the sample replayable by recomputing draws and observations from an edited seed, but the edited seed contains a non-digit character. The generic sampler and stopping layers can still be made internally consistent; the jurisdiction adapter rejects the package under **rla_jurisdiction_adapter**.

The public-tool fixture is **california-rla**. It adds a California-style method id, ballot manifest format id, audit software id, and public source URL to the statistical fixture. The adapter verifies that this public tooling metadata is present and shaped like a reproducible public audit record.

The public-tool negative fixture is **bad-california-rla**. It keeps the audit otherwise

internally consistent, but changes the source field to a non-URL identifier. The package fails under `rla_jurisdiction_adapter`.

5 Boundaries

This slice verifies sample selection and a deliberately small family of stopping contracts. The newest method, `comparison-margin-threshold-v1`, is a deterministic comparison-margin risk-estimate check: it binds sample size, reported margin, discrepancy count, declared risk estimate, and risk limit into one replayable equation. It is not a Kaplan-Markov, ballot-polling, or full statutory ballot-level comparison risk measure. It also does not decide whether a jurisdiction’s legal RLA method permits replacement sampling, modulo reduction, or a particular manifest construction.

Those boundaries are deliberate. V.7 now has a replayable sampler surface, a minimal observation-to-status surface, explicit discrepancy taxonomy, bound margin metadata, and a first named statistical-risk surface that later work can reuse. The next layer should add Kaplan-Markov or jurisdiction-specific stopping math, escalation workflows, and fuller state-specific sampler adapters. The Colorado- and California-style adapters landed here are only record-shape, seed-format, and public-tool metadata contracts for synthetic fixtures; they are bridges toward statutory review, not legal certification claims.

The privacy boundary also matters. A public RLA sample can reveal ballot-level records selected for audit. RCOUNT can make the sample reproducible, but jurisdictions still need publication rules for small batches, rare ballot styles, and any join between CVRs and voter-facing inclusion tokens.

6 Conclusion

RCOUNT now has an implemented replay contract for the first half of an RLA: the sample list. It also has a first observation and stopping-status contract. The verifier recomputes the contest population hash and every draw from public seed material, then checks that observed marks support the declared pass-or-escalate status. When discrepancies are present, the verifier also checks that the declared discrepancy taxonomy matches the computed mismatch. Reported margin metadata is no longer free text; it is recomputed from the public jurisdiction summary. The first named statistical method also recomputes a declared risk estimate before accepting a stopping claim. Colorado- and California-style adapters now show where jurisdiction-specific record contracts attach without confusing those contracts with legal certification.

The remaining V.7 work is to add legally specified statistical stopping methods, especially Kaplan-Markov or state-specific adapters, on top of the same audit surface. With sampler replay and observations in place, that layer can focus on jurisdictional risk calculations and escalation without also carrying the burden of proving how the sample was chosen, what type of mismatch occurred, or which public totals define the margin.

References

- Gio Della-Libera. Rcount overview: Reproducible election count packages. Technical report, Apportionment Research, 2026a.
- Gio Della-Libera. Ballot manifest verification. Technical report, Apportionment Research, 2026b.
- Gio Della-Libera. Cvr-to-summary reconciliation. Technical report, Apportionment Research, 2026c.